

MONTH 1-2

GOAL

The final system will have hundreds of vehicles each running a small AI model that watches the vehicle's CAN bus traffic in real time. If the traffic looks abnormal, like an attacker is injecting fake messages, the model raises an alarm. What makes this special is that these vehicles also collaborate with each other to improve their models, but they do this without sharing their raw data (that's the Federated Learning part), and without even sharing their model updates in readable form (that's the FHE part/path).

(Base papers: DCANFL and FedIoV)

Dataset Selection:

- **SynCAN** is the first dataset to start with. "Syn" stands for synthetic — the CAN signals in this dataset are simulated but carefully designed to mimic real vehicle behavior. The advantage is that it is clean, well-structured, and comes with clear labels for five attack types: flooding, fuzzing, replay, suppression (called plateau), and continuous (a type of injection). The signals are already decoded into meaningful numerical values. This is the best dataset for getting started and the one used in the DCANFL paper.
- **Car-Hacking Dataset** is from real Hyundai vehicles and contains raw CAN frame data (hex IDs and hex payloads). It covers spoofing, fuzzing, and replay attacks. The challenge is that the data is in raw format and needs parsing. This is a good second dataset to work with once the pipeline is running on SynCAN.
- **X-CANIDS** is the most realistic and also the most complex. It contains real CAN traffic from a production vehicle, and the data has been deserialized into signal-level representations using a DBC file (a database file that defines what each CAN ID means and how to decode its payload bytes). It covers five attack categories. This is the dataset used in both papers. It is more challenging because of its complexity, we will come back to it after getting results on SynCAN.

The deliverables that the company expects from the IDS track at the end of month 1–2 are two specific documents: a **Threat Model Document** and a **Baseline Architecture Design**.

1. The **Threat Model Document**: a written report of four to six pages (Or how many pages you want depending on you) covering the attack types you're detecting, the adversary model (what the attacker can and cannot

do), the trust assumptions, and the detection goals (target AUC and F1 values).

2. The **Baseline Architecture Design** should document the autoencoder architecture you chose, number of layers, layer sizes, activation functions, loss function, windowing strategy, and normalization approach.

Deliverable 1

The threat model document needs to answer these specific questions:

What is the system being protected? The CAN bus of a connected vehicle participating in a federated learning network. Each vehicle runs a local IDS model and occasionally shares model updates (in encrypted form, once your FHE layer is added) with an aggregation server.

Who is the adversary? There are two separate adversary types they need to define. The first is an in-vehicle attacker — someone who has gained access to the CAN bus (physically or remotely) and is injecting, replaying, suppressing, or fuzzing messages. This is what the IDS model itself detects. The second is a network-level adversary — someone who intercepts FL communications and tries to extract private vehicle data from the model updates. This second threat is what the FHE work addresses, so the IDS path doesn't need to solve it, but need to mention it so the system design is complete.

What can the adversary do? For the in-vehicle attacker: they can inject arbitrary CAN frames, replay captured frames, block messages, and flood the bus. For the network adversary: they can eavesdrop on model updates sent to the aggregation server (this is where FHE protects).

What can the adversary NOT do? They cannot modify the IDS model itself (that would require physical access to the ECU). They cannot break the FHE encryption (that's mathematically guaranteed). They don't know the detection threshold of the IDS.

What is the trust model? Each vehicle's local data stays on the vehicle — it is never shared. The aggregation server is "honest but curious" — it follows the protocol correctly but might try to infer things from what it receives. This is why FHE is needed: even an honest server should not be able to see plaintext model weights.

What are the detection goals? High recall (few missed attacks — a missed attack in a vehicle is dangerous) and acceptable precision (few false alarms — too many false alarms and the driver ignores the system). A possible target could be: F1 score above 0.85 and AUC above 0.90 on each attack type, based on the numbers from the base papers.

Deliverable 2

There are two broad approaches:

- 1) The first approach is **supervised classification**. You train a model to distinguish between "normal" and "attack" traffic by showing it labeled examples of both. This typically gives very high accuracy on known attack types because the model has directly learned what attacks look like. The problem is that it requires labeled attack data — someone had to actually generate and label the attacks — and it may fail completely on new attack types it has never seen (zero-day attacks).
- 2) The second approach is **unsupervised anomaly detection**. You train a model only on normal traffic, teaching it to reconstruct normal CAN signals accurately. At detection time, you run every sample through the model and measure the reconstruction error. Normal traffic should have low error. Abnormal traffic — attacks, anomalies, anything the model hasn't seen during training — should have high error because the model doesn't know how to reconstruct unusual patterns. The advantage is that you don't need labeled attack data for training, and the model can detect attack types it has never seen before. The disadvantage is that subtle attacks (like masquerade) might look normal enough that the reconstruction error doesn't spike.

Use the unsupervised autoencoder approach, following the CANet architecture from the DCANFL paper.

The recommendation, and the justification for it (You might need to explain in the presentation about why you have chosen this approach):

1. First, in real vehicle deployments, you will almost never have labeled attack data. Automakers don't intentionally attack their own vehicles to generate training labels. The unsupervised approach is what would actually be deployed in the real world.
2. Second, the autoencoder approach is well-validated — the DCANFL paper uses it and gets AUC above 0.95 on most attack types.
3. Third, it aligns with how federated learning works naturally. In FL, each vehicle trains on its own local data. If that training data is only normal traffic (which it would be in a real deployment — you train on what you observe before any attack happens), then an unsupervised model is the only viable option.
4. Fourth, and most importantly for your project, the autoencoder learns a compressed representation of normal CAN signal patterns. When you later add FHE and need to encrypt model weights, the autoencoder is a relatively small model, which keeps the FHE overhead manageable.

The CAN Bus architecture

A vehicle's CAN (Controller Area Network) bus is essentially the internal nervous system of a car. Every Electronic Control Unit (ECU) — the brakes, the engine, the steering, the airbags — communicates with every other ECU over this shared bus by sending small messages called CAN frames. Each frame has an identifier (an 11-bit ID in hex) and a data payload (up to 8 bytes). These messages fly back and forth thousands of times per second.

The problem is that CAN was designed in 1983 for closed systems where nobody expected attackers. There is no authentication, no encryption, no way to verify that a message is actually coming from the ECU it claims to be from. So if an attacker gains access — through the OBD-II port, the infotainment system, a Bluetooth exploit, or anything else — they can inject fake CAN messages and the bus has no way to reject them.

The attacks that we need to detect fall into these categories:

1. A **Flooding attack** means an attacker is sending an enormous number of messages in a short time, overwhelming the bus. The signature is an unnaturally high message frequency for a particular ID.
2. A **Fuzzing attack** means an attacker is sending messages with random data payloads, trying to find something that causes unexpected behaviour. The signature is messages with valid IDs but random, nonsensical data values that don't match real physical states.
3. A **Replay attack** means an attacker has previously recorded legitimate CAN traffic and is now replaying it at the wrong time. The signature is valid-looking messages that appear in the wrong temporal context.
4. A **Suppression attack** means an attacker is blocking legitimate messages from reaching their destination. The signature is an abnormally low message frequency for a particular ID.
5. A **Masquerade attack** is the hardest to detect — the attacker sends carefully crafted messages that look realistic but contain subtly wrong data values. For example, injecting a fake engine RPM reading that's slightly off.

Just go through it once, there might be other attacks possible.

The survey paper by Jo and Choi (2022) cited in the DCANFL paper is a good starting point.

Environment Setup

Install:

- Python 3.10 / 3.11

Create:

- Virtual environment (venv / conda)

Install required libraries: (Whatever is needed)

- PyTorch
- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn
- Jupyter Notebook

Initially, You need to tell me one number: the approximate parameter counts of the model architecture you've chosen. I will be needing that to work on the FHE part. Take time to explore it. Even before the model is trained, you can instantiate it in PyTorch and run `sum(p.numel() for p in model.parameters())` to count parameters. Message me this number as soon as you have it.

Next, the second interaction happens at the start of month 3, when you have your final model architecture decided and the architecture file available. We will discuss this later.